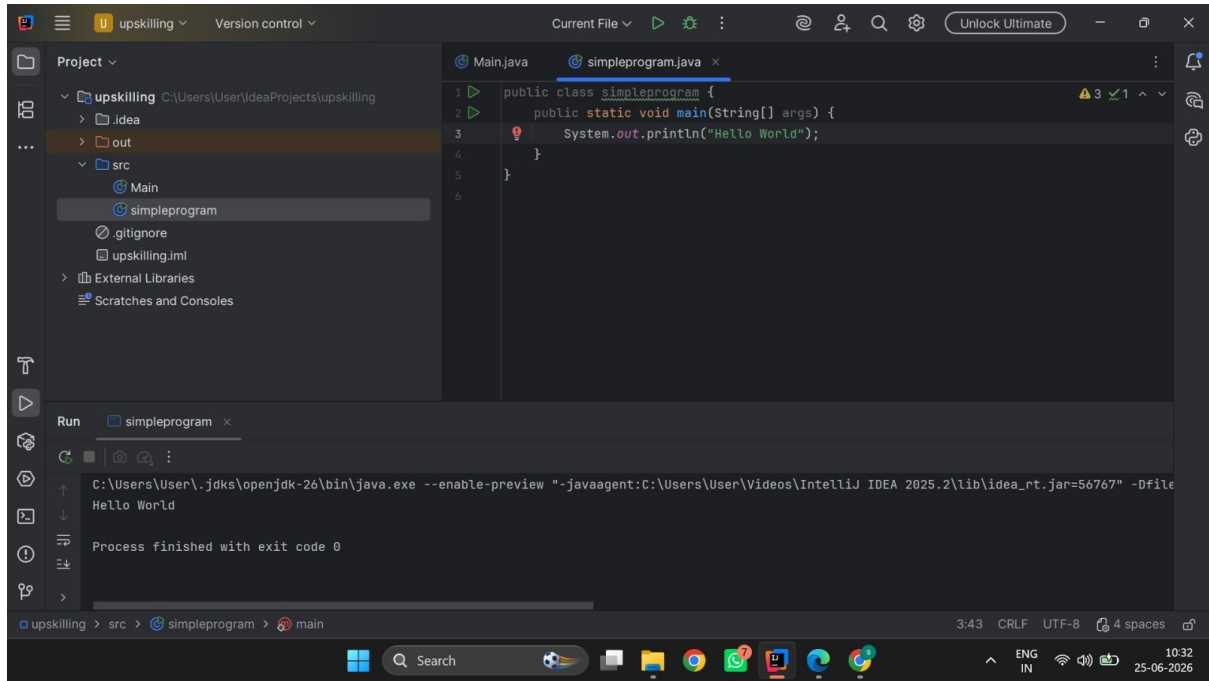


Core Java Exercises

Hello World Program

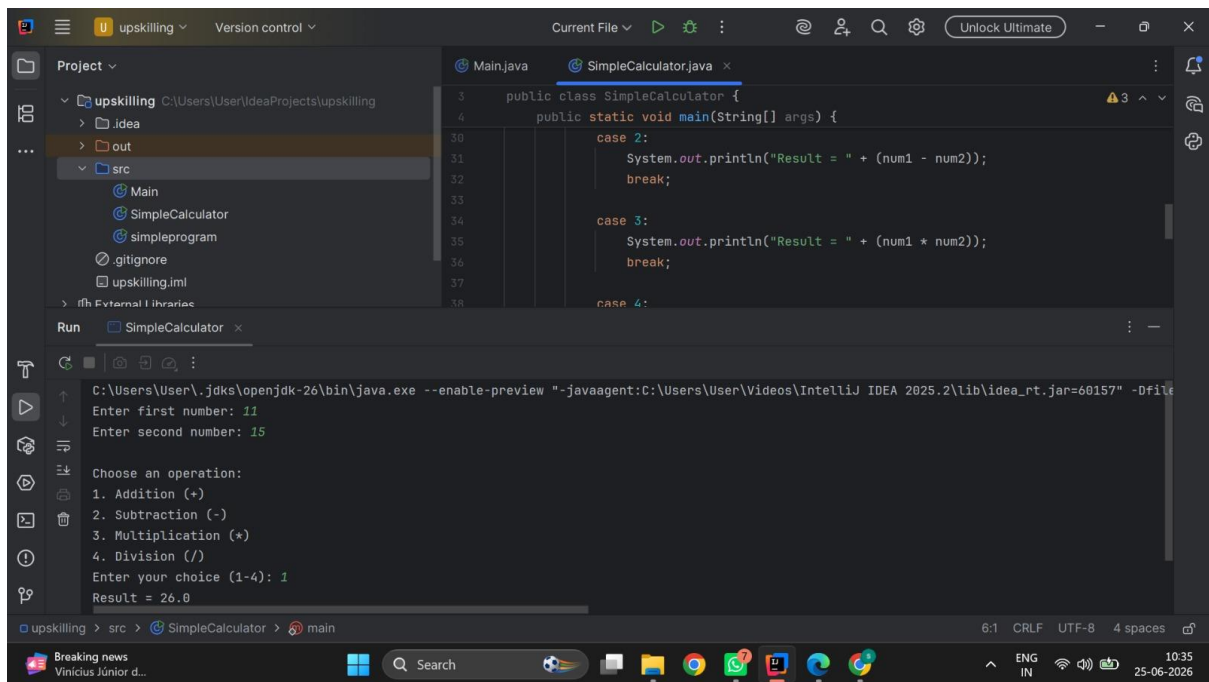


The screenshot shows the IntelliJ IDEA interface with a project named 'upskilling'. The 'src' directory contains a package 'simpleprogram' with a class 'Main'. The 'Main.java' file is open, showing the following code:

```
1 public class simpleprogram {
2     public static void main(String[] args) {
3         System.out.println("Hello World");
4     }
5 }
6
```

The Run window at the bottom shows the command executed: `C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=56767" -Dfile.encoding=UTF-8`. The output is 'Hello World'.

Simple Calculator

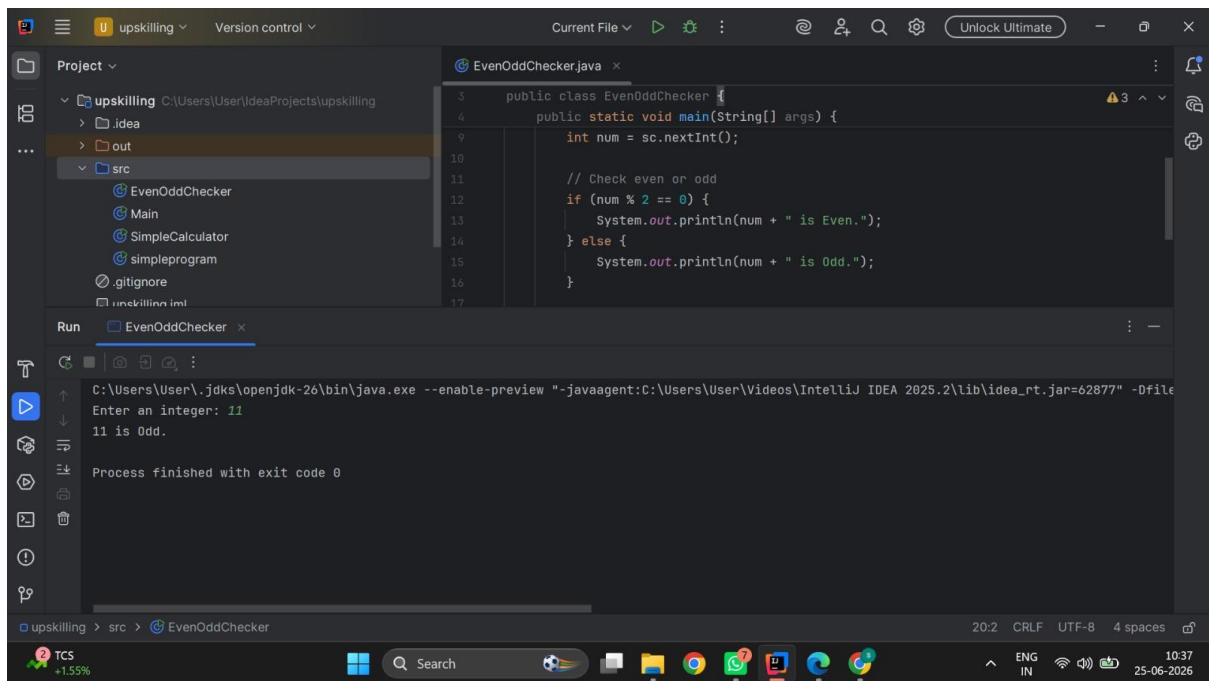


The screenshot shows the IntelliJ IDEA interface with a project named 'upskilling'. The 'src' directory contains a package 'SimpleCalculator' with a class 'SimpleCalculator'. The 'SimpleCalculator.java' file is open, showing the following code:

```
3 public class SimpleCalculator {
4     public static void main(String[] args) {
5         // ... (code for input and calculation)
6     }
7 }
8
```

The Run window at the bottom shows the command executed: `C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=60157" -Dfile.encoding=UTF-8`. The output shows the program prompts for two numbers (11 and 15) and an operation (1 for Addition), resulting in 'Result = 26.0'.

Even or Odd Checker

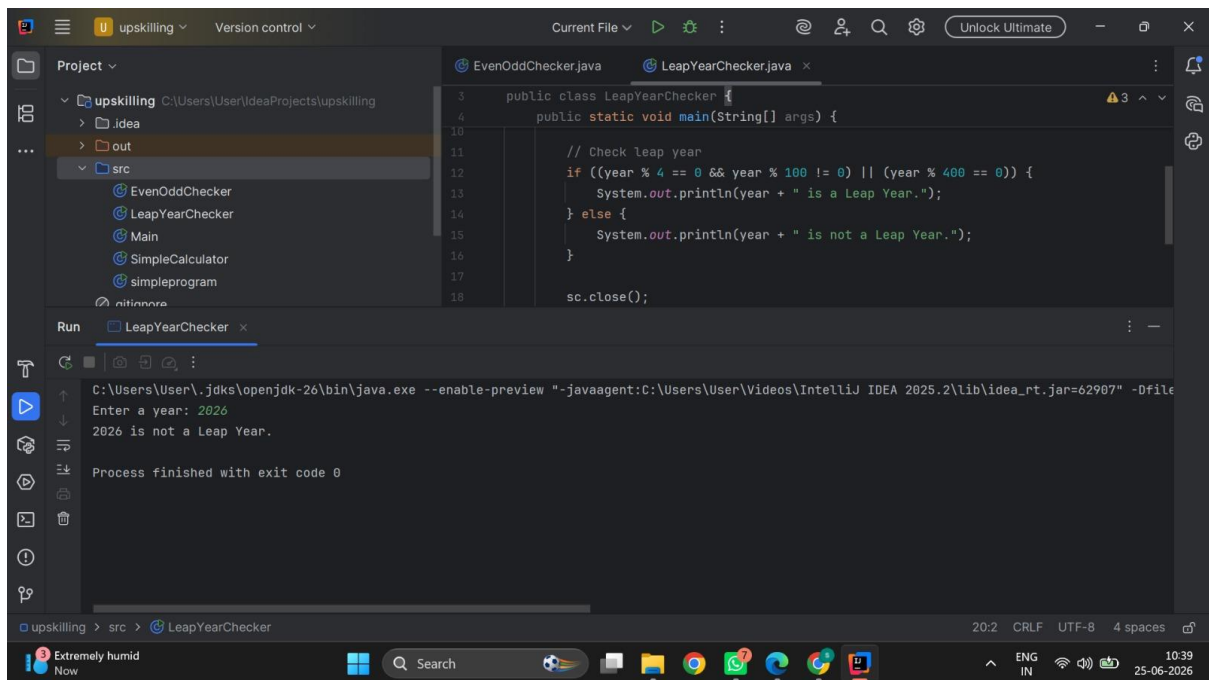


The screenshot shows the IntelliJ IDEA interface with the 'EvenOddChecker.java' file open. The code defines a public class 'EvenOddChecker' with a static 'main' method that takes a 'String[] args' parameter. Inside the 'main' method, an integer 'num' is read from the user using 'sc.nextInt()'. A comment indicates the purpose is to 'Check even or odd'. An 'if' statement checks if 'num % 2 == 0'. If true, it prints 'num is Even.'; otherwise, it prints 'num is Odd.'.

```
3 public class EvenOddChecker {
4     public static void main(String[] args) {
9         int num = sc.nextInt();
10
11         // Check even or odd
12         if (num % 2 == 0) {
13             System.out.println(num + " is Even.");
14         } else {
15             System.out.println(num + " is Odd.");
16         }
17     }
18 }
```

The 'Run' tab shows the execution output: 'Enter an integer: 11' followed by '11 is Odd.' and 'Process finished with exit code 0'.

Leap Year Checker

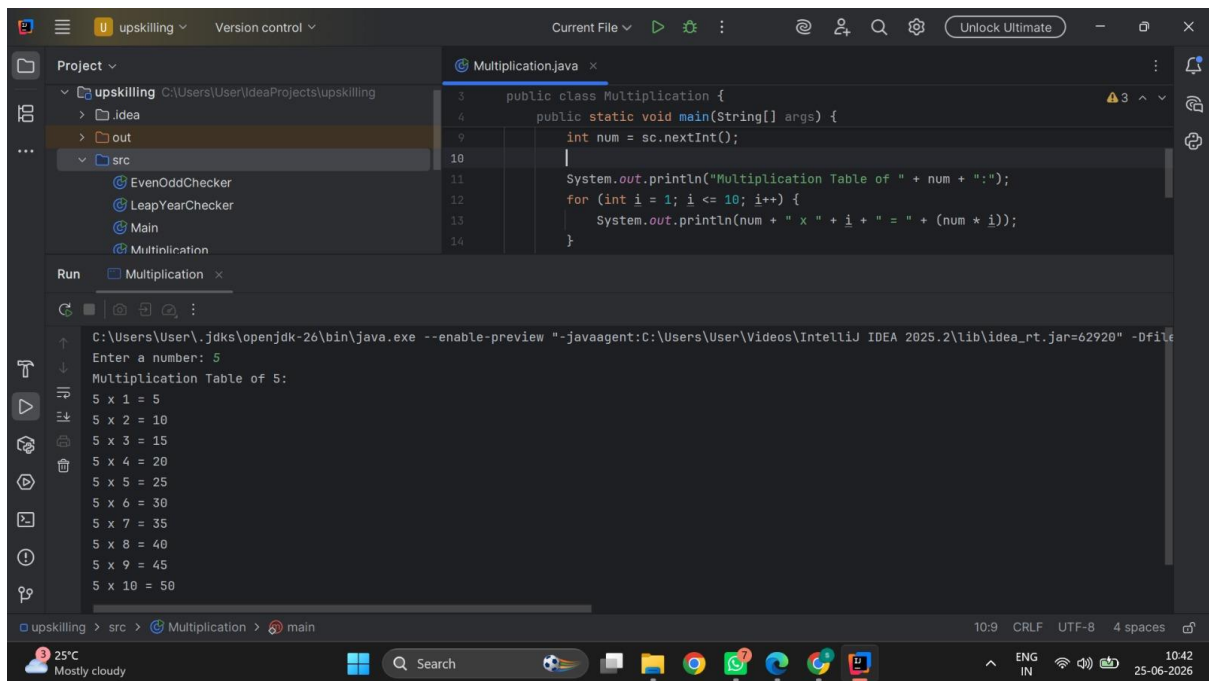


The screenshot shows the IntelliJ IDEA interface with the 'LeapYearChecker.java' file open. The code defines a public class 'LeapYearChecker' with a static 'main' method that takes a 'String[] args' parameter. Inside the 'main' method, a year is read from the user using 'sc.nextInt()'. A comment indicates the purpose is to 'Check leap year'. An 'if' statement checks if '(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)'. If true, it prints 'year is a Leap Year.'; otherwise, it prints 'year is not a Leap Year.'.

```
3 public class LeapYearChecker {
4     public static void main(String[] args) {
10
11         // Check leap year
12         if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
13             System.out.println(year + " is a Leap Year.");
14         } else {
15             System.out.println(year + " is not a Leap Year.");
16         }
17     }
18     sc.close();
19 }
```

The 'Run' tab shows the execution output: 'Enter a year: 2026' followed by '2026 is not a Leap Year.' and 'Process finished with exit code 0'.

Multiplication Table



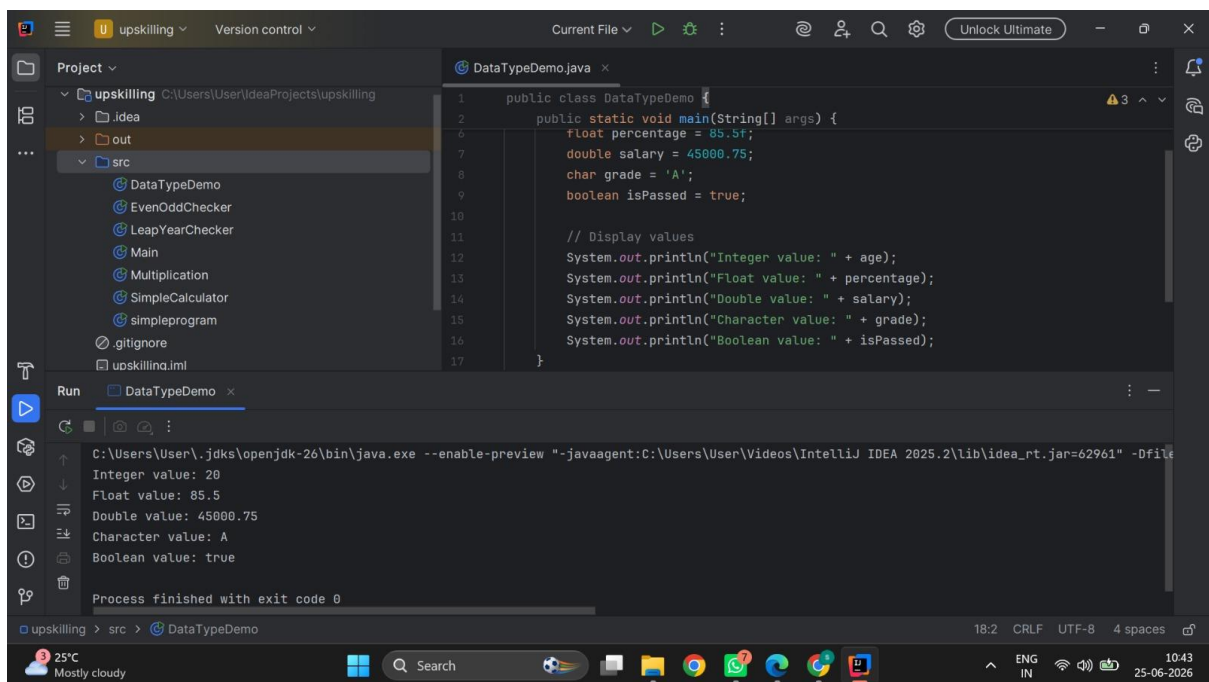
The screenshot shows the IntelliJ IDEA interface with the 'upskilling' project. The 'Project' view on the left shows the file structure: 'idea', 'out', and 'src'. The 'src' folder contains 'EvenOddChecker', 'LeapYearChecker', 'Main', and 'Multiplication'. The 'Multiplication.java' file is open in the editor, showing the following code:

```
3 public class Multiplication {
4     public static void main(String[] args) {
5         int num = sc.nextInt();
6         System.out.println("Multiplication Table of " + num + ":");
7         for (int i = 1; i <= 10; i++) {
8             System.out.println(num + " x " + i + " = " + (num * i));
9         }
10    }
```

The 'Run' view at the bottom shows the execution output:

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=62920" -Dfile.encoding=UTF-8
Enter a number: 5
Multiplication Table of 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Data Type Demonstration



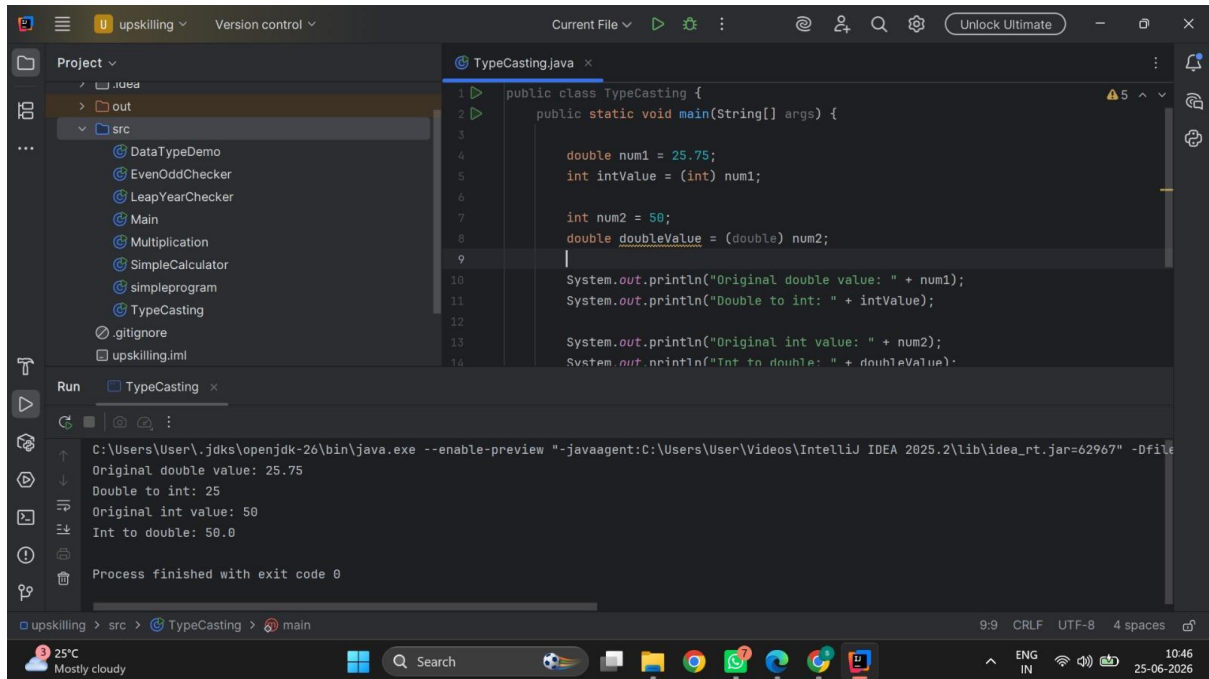
The screenshot shows the IntelliJ IDEA interface with the 'upskilling' project. The 'Project' view on the left shows the file structure: 'idea', 'out', and 'src'. The 'src' folder contains 'DataTypeDemo', 'EvenOddChecker', 'LeapYearChecker', 'Main', 'Multiplication', 'SimpleCalculator', and 'simpleprogram'. The 'DataTypeDemo.java' file is open in the editor, showing the following code:

```
1 public class DataTypeDemo {
2     public static void main(String[] args) {
3         float percentage = 85.5f;
4         double salary = 45000.75;
5         char grade = 'A';
6         boolean isPassed = true;
7
8         // Display values
9         System.out.println("Integer value: " + age);
10        System.out.println("Float value: " + percentage);
11        System.out.println("Double value: " + salary);
12        System.out.println("Character value: " + grade);
13        System.out.println("Boolean value: " + isPassed);
14    }
```

The 'Run' view at the bottom shows the execution output:

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=62961" -Dfile.encoding=UTF-8
Integer value: 20
Float value: 85.5
Double value: 45000.75
Character value: A
Boolean value: true
Process finished with exit code 0
```

Type Casting Example

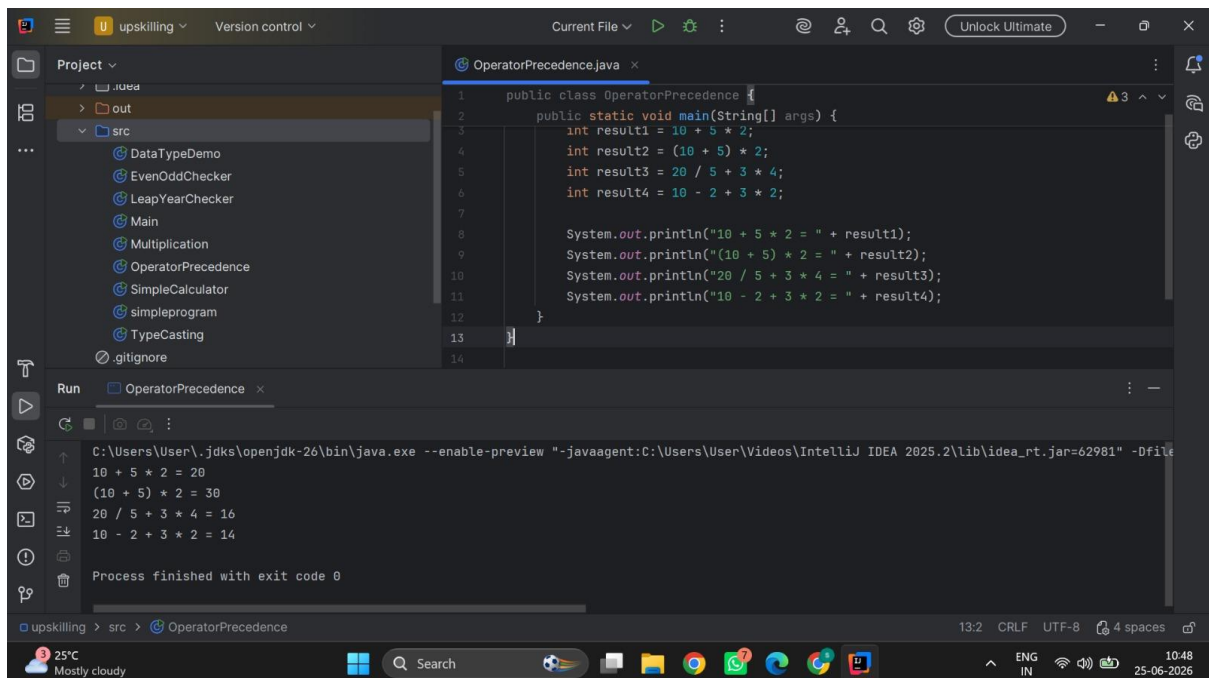


```
1 public class TypeCasting {
2     public static void main(String[] args) {
3
4         double num1 = 25.75;
5         int intValue = (int) num1;
6
7         int num2 = 50;
8         double doubleValue = (double) num2;
9
10        System.out.println("Original double value: " + num1);
11        System.out.println("Double to int: " + intValue);
12
13        System.out.println("Original int value: " + num2);
14        System.out.println("Int to double: " + doubleValue);
15    }
16 }
```

Run TypeCasting

```
C:\Users\User\jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=62967" -Dfile.encoding=UTF-8
Original double value: 25.75
Double to int: 25
Original int value: 50
Int to double: 50.0
Process finished with exit code 0
```

Operator Precedence

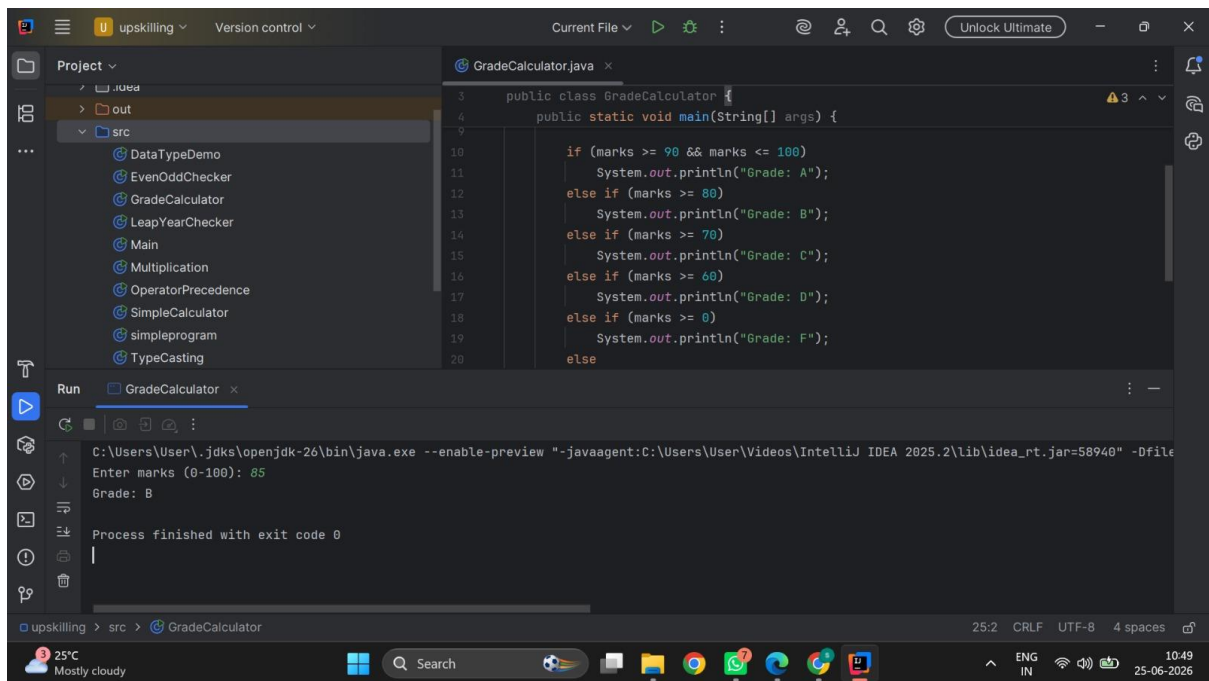


```
1 public class OperatorPrecedence {
2     public static void main(String[] args) {
3         int result1 = 10 + 5 * 2;
4         int result2 = (10 + 5) * 2;
5         int result3 = 20 / 5 + 3 * 4;
6         int result4 = 10 - 2 + 3 * 2;
7
8         System.out.println("10 + 5 * 2 = " + result1);
9         System.out.println("(10 + 5) * 2 = " + result2);
10        System.out.println("20 / 5 + 3 * 4 = " + result3);
11        System.out.println("10 - 2 + 3 * 2 = " + result4);
12    }
13 }
14 }
```

Run OperatorPrecedence

```
C:\Users\User\jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=62981" -Dfile.encoding=UTF-8
10 + 5 * 2 = 20
(10 + 5) * 2 = 30
20 / 5 + 3 * 4 = 16
10 - 2 + 3 * 2 = 14
Process finished with exit code 0
```

Grade Calculator



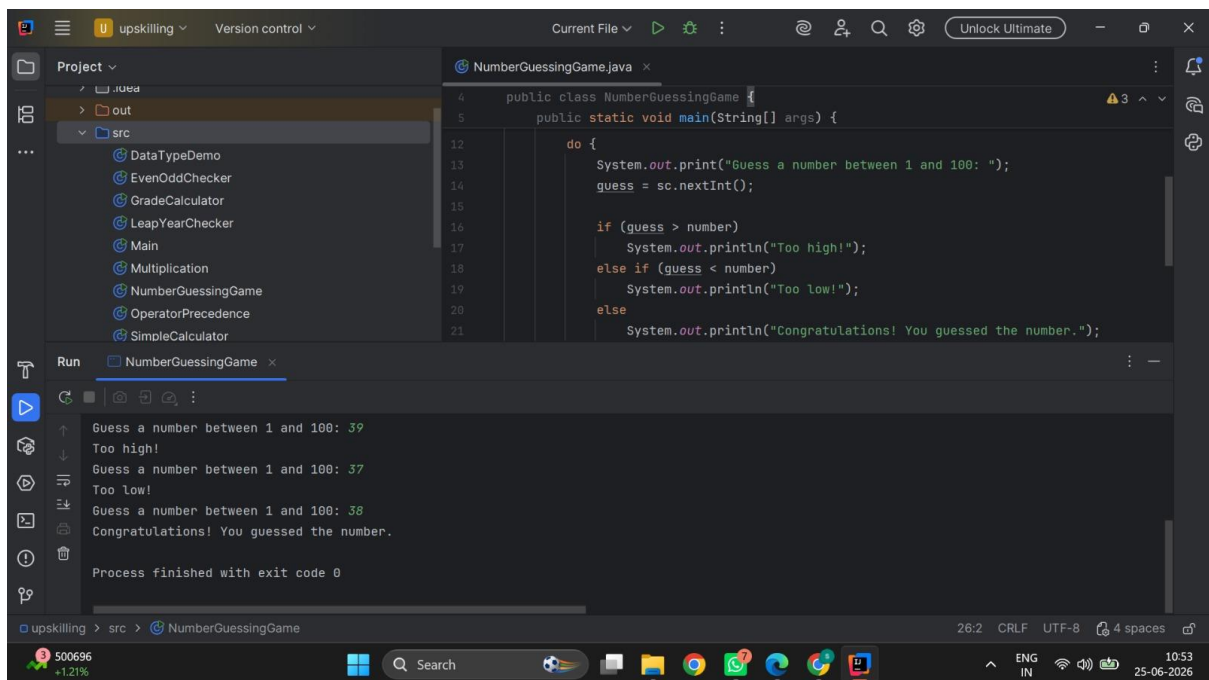
The screenshot shows the IntelliJ IDEA IDE with the 'GradeCalculator.java' file open. The code defines a public class 'GradeCalculator' with a static method 'main' that takes an array of strings as input. It prompts the user to enter marks (0-100) and then uses a series of 'if-else' statements to determine the corresponding grade (A, B, C, D, E, F) based on the input marks. The 'Run' button is visible, and the output window shows the execution results.

```
public class GradeCalculator {  
    public static void main(String[] args) {  
        // ...  
        if (marks >= 90 && marks <= 100)  
            System.out.println("Grade: A");  
        else if (marks >= 80)  
            System.out.println("Grade: B");  
        else if (marks >= 70)  
            System.out.println("Grade: C");  
        else if (marks >= 60)  
            System.out.println("Grade: D");  
        else if (marks >= 50)  
            System.out.println("Grade: E");  
        else  
            System.out.println("Grade: F");  
    }  
}
```

Run Output:

```
C:\Users\User\jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=58940" -Dfile.encoding=UTF-8  
Enter marks (0-100): 85  
Grade: B  
  
Process finished with exit code 0
```

Number Guessing Game



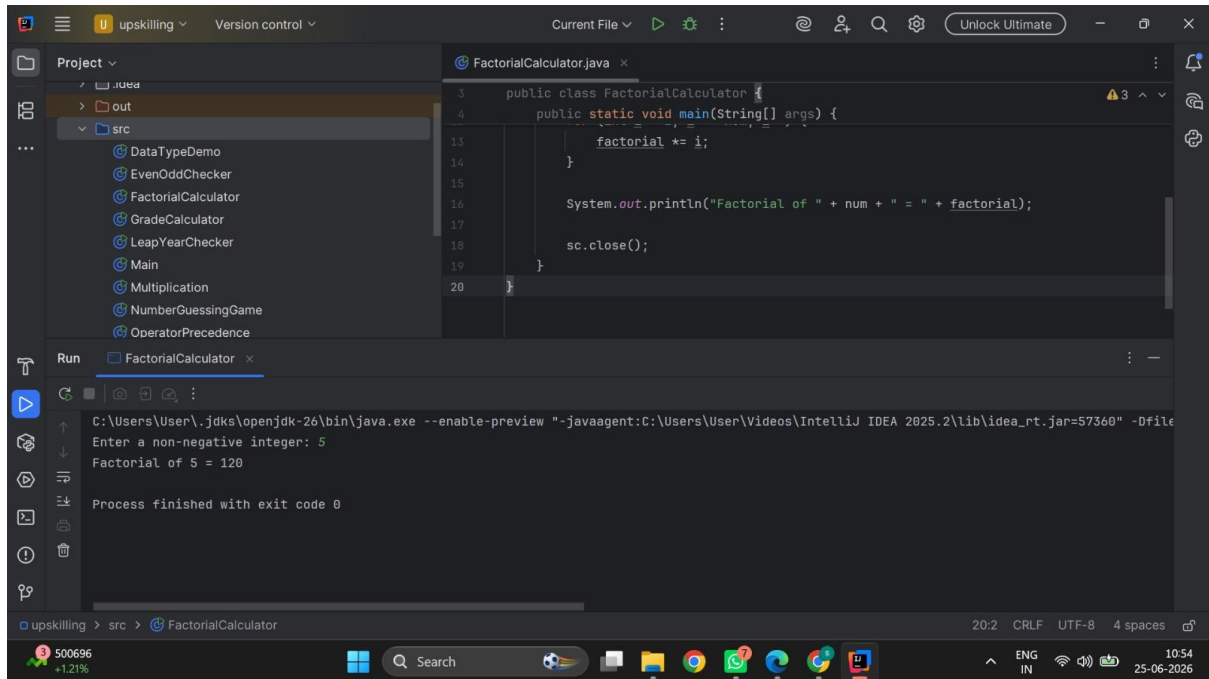
The screenshot shows the IntelliJ IDEA IDE with the 'NumberGuessingGame.java' file open. The code defines a public class 'NumberGuessingGame' with a static method 'main' that takes an array of strings as input. It prompts the user to guess a number between 1 and 100. It uses a 'do-while' loop to repeatedly prompt the user until they guess the correct number. The 'Run' button is visible, and the output window shows the execution results.

```
public class NumberGuessingGame {  
    public static void main(String[] args) {  
        // ...  
        do {  
            System.out.print("Guess a number between 1 and 100: ");  
            guess = sc.nextInt();  
            if (guess > number)  
                System.out.println("Too high!");  
            else if (guess < number)  
                System.out.println("Too low!");  
            else  
                System.out.println("Congratulations! You guessed the number.");  
        } while (guess != number);  
    }  
}
```

Run Output:

```
Guess a number between 1 and 100: 39  
Too high!  
Guess a number between 1 and 100: 37  
Too low!  
Guess a number between 1 and 100: 38  
Congratulations! You guessed the number.  
  
Process finished with exit code 0
```

Factorial Calculator



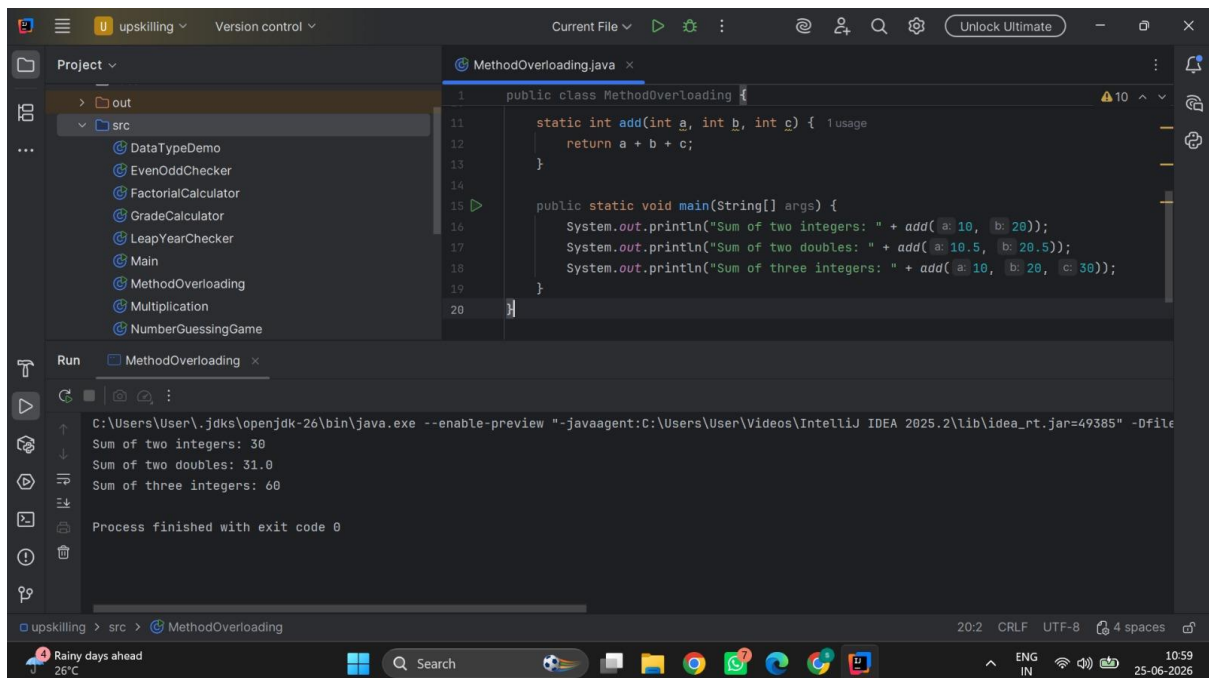
The screenshot shows the IntelliJ IDEA interface with the `FactorialCalculator.java` file open. The code defines a `FactorialCalculator` class with a `main` method that calculates the factorial of a user-input integer. The `main` method uses a `Scanner` to read input, calculates the factorial using a loop, and prints the result.

```
3 public class FactorialCalculator {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6         int num = sc.nextInt();
7         long factorial = 1;
8         for (int i = 1; i <= num; i++) {
9             factorial *= i;
10        }
11        System.out.println("Factorial of " + num + " = " + factorial);
12        sc.close();
13    }
14}
```

The Run window shows the execution output:

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=57360" -Dfile.encoding=UTF-8
Enter a non-negative integer: 5
Factorial of 5 = 120
Process finished with exit code 0
```

Method Overloading



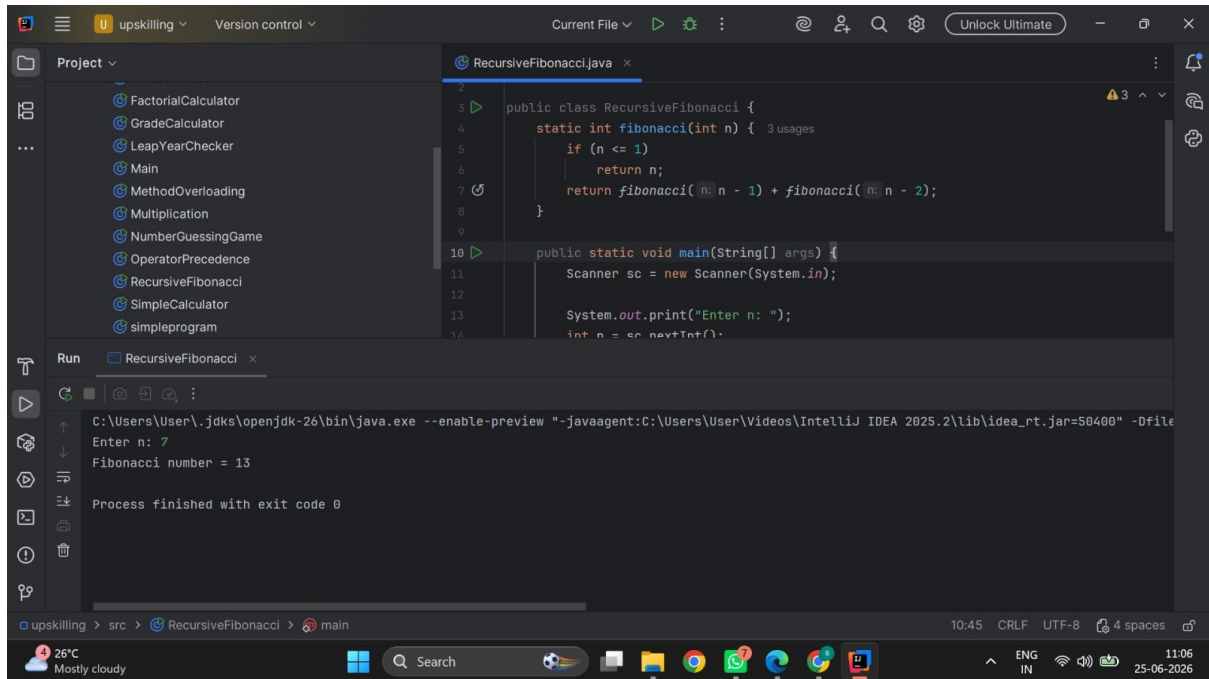
The screenshot shows the IntelliJ IDEA interface with the `MethodOverloading.java` file open. The code defines a `MethodOverloading` class with a `main` method that demonstrates method overloading by calling the `add` method with different parameter lists. The `add` method is overloaded to handle different types of numbers (integers, doubles, and a mix of integers and doubles).

```
1 public class MethodOverloading {
2     static int add(int a, int b, int c) {
3         return a + b + c;
4     }
5     public static void main(String[] args) {
6         System.out.println("Sum of two integers: " + add(10, 20));
7         System.out.println("Sum of two doubles: " + add(10.5, 20.5));
8         System.out.println("Sum of three integers: " + add(10, 20, 30));
9     }
10 }
```

The Run window shows the execution output:

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=49385" -Dfile.encoding=UTF-8
Sum of two integers: 30
Sum of two doubles: 31.0
Sum of three integers: 60
Process finished with exit code 0
```


Recursive Fibonacci



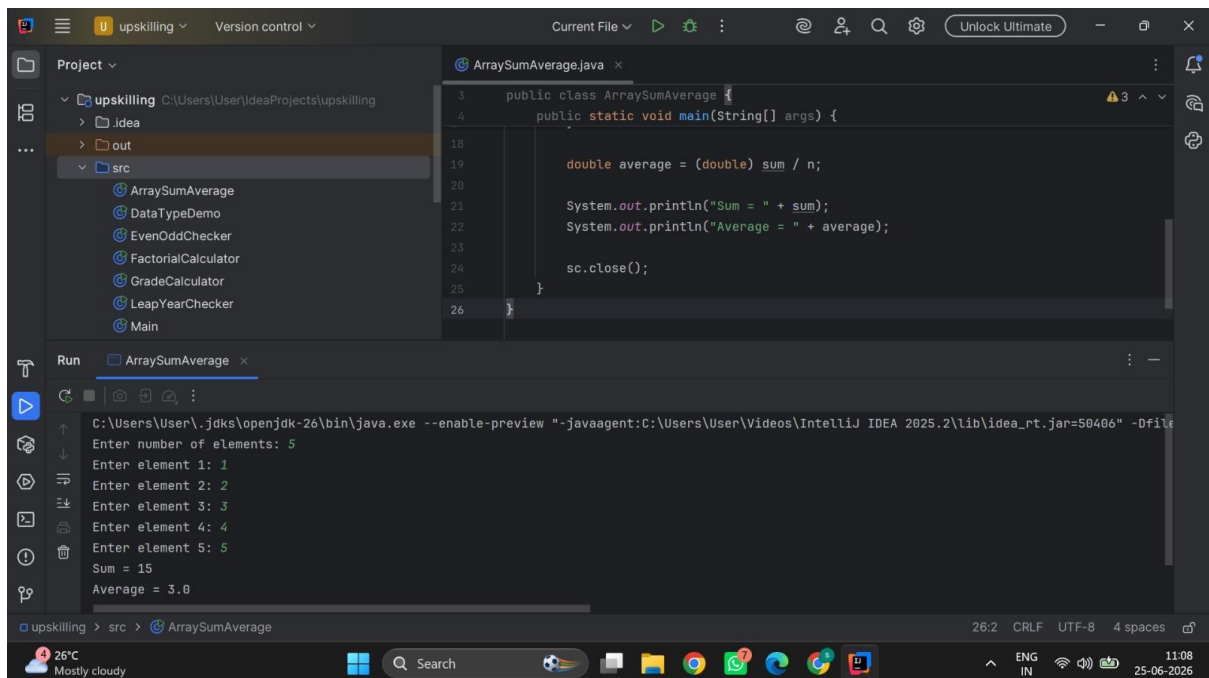
The screenshot shows the IntelliJ IDEA IDE with the 'RecursiveFibonacci.java' file open. The code defines a recursive method to calculate the Fibonacci sequence. The 'Run' window shows the program's execution, where the user enters '7' and the output is 'Fibonacci number = 13'.

```
1 public class RecursiveFibonacci {
2
3     static int fibonacci(int n) { 3 usages
4         if (n <= 1)
5             return n;
6         return fibonacci(n - 1) + fibonacci(n - 2);
7     }
8
9     public static void main(String[] args) {
10         Scanner sc = new Scanner(System.in);
11
12         System.out.print("Enter n: ");
13         int n = sc.nextInt();
14     }
15 }
```

Run: RecursiveFibonacci x

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50400" -Dfile.encoding=UTF-8
Enter n: 7
Fibonacci number = 13
Process finished with exit code 0
```

Array Sum and Average



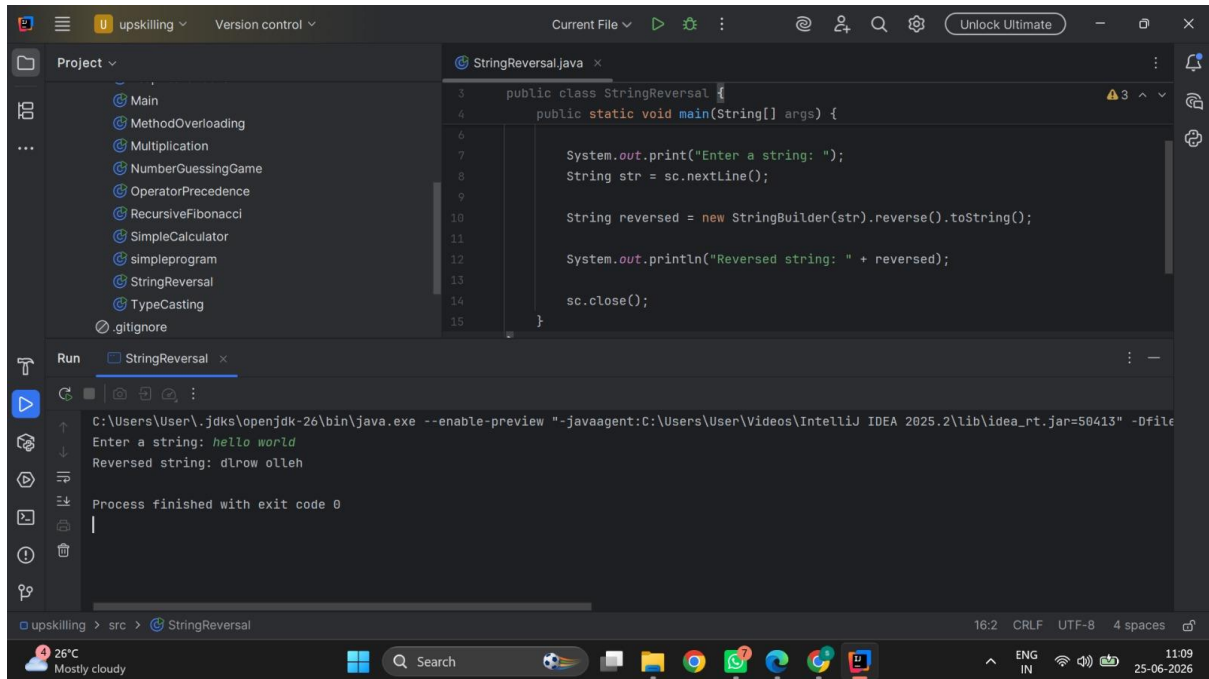
The screenshot shows the IntelliJ IDEA IDE with the 'ArraySumAverage.java' file open. The code defines a method to calculate the sum and average of an array of integers. The 'Run' window shows the program's execution, where the user enters '5' for the number of elements, followed by five elements (1, 2, 3, 4, 5), resulting in a sum of 15 and an average of 3.0.

```
1 public class ArraySumAverage {
2     public static void main(String[] args) {
3
4         double sum = 0;
5         double average = (double) sum / n;
6
7         System.out.println("Sum = " + sum);
8         System.out.println("Average = " + average);
9
10        sc.close();
11    }
12 }
```

Run: ArraySumAverage x

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50400" -Dfile.encoding=UTF-8
Enter number of elements: 5
Enter element 1: 1
Enter element 2: 2
Enter element 3: 3
Enter element 4: 4
Enter element 5: 5
Sum = 15
Average = 3.0
```

String Reversal



The screenshot shows the IntelliJ IDEA IDE with a project named 'upskilling'. The 'Project' view on the left lists various Java programs, including 'StringReversal'. The main editor displays the code for 'StringReversal.java'.

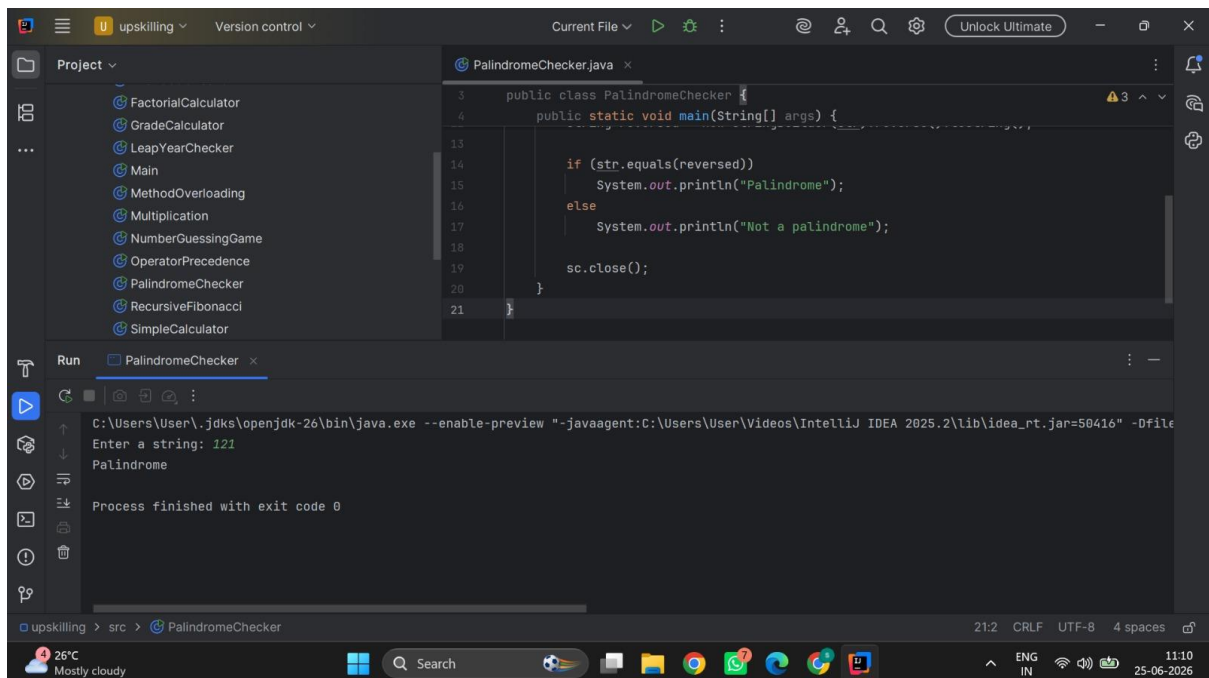
```
3 public class StringReversal {
4     public static void main(String[] args) {
5
6         System.out.print("Enter a string: ");
7         String str = sc.nextLine();
8
9         String reversed = new StringBuilder(str).reverse().toString();
10
11        System.out.println("Reversed string: " + reversed);
12
13        sc.close();
14    }
15 }
```

The 'Run' window at the bottom shows the execution output:

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50413" -Dfile.encoding=UTF-8
Enter a string: hello world
Reversed string: dlrow olleh
Process finished with exit code 0
```

The status bar at the bottom indicates the file is in 'src > StringReversal' with encoding 'UTF-8' and 4 spaces.

Palindrome Checker



The screenshot shows the IntelliJ IDEA IDE with a project named 'upskilling'. The 'Project' view on the left lists various Java programs, including 'PalindromeChecker'. The main editor displays the code for 'PalindromeChecker.java'.

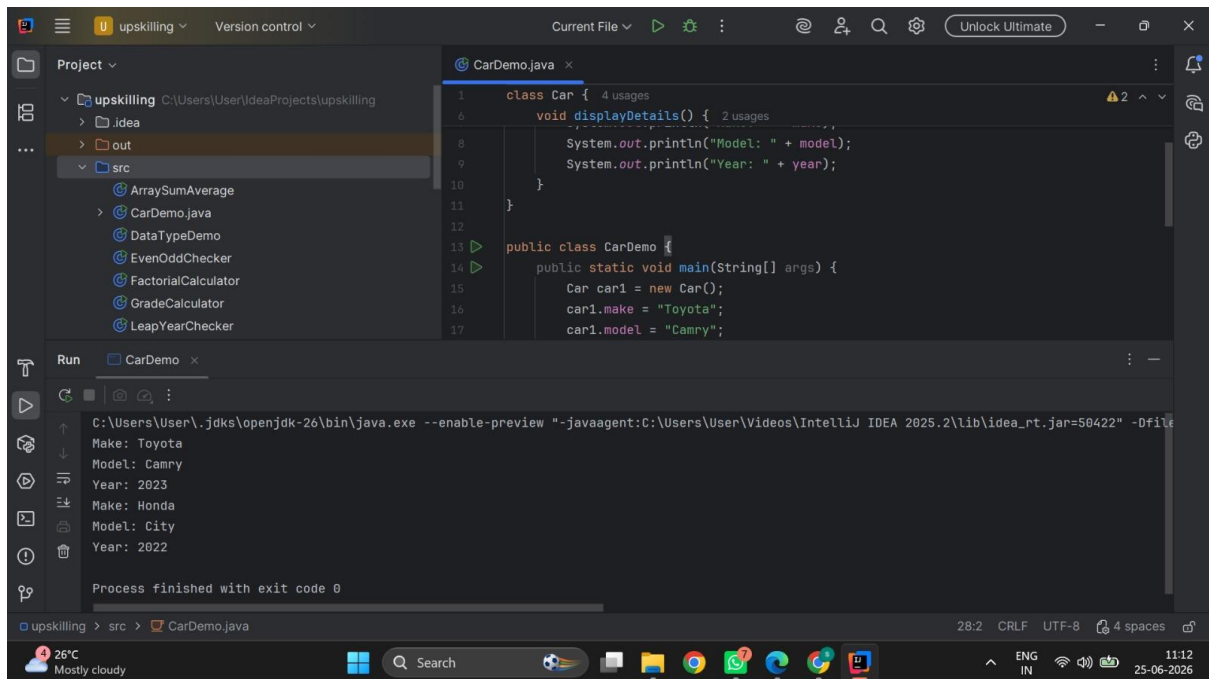
```
3 public class PalindromeChecker {
4     public static void main(String[] args) {
5
6         System.out.print("Enter a string: ");
7         String str = sc.nextLine();
8
9         String reversed = new StringBuilder(str).reverse().toString();
10
11        if (str.equals(reversed))
12            System.out.println("Palindrome");
13        else
14            System.out.println("Not a palindrome");
15
16        sc.close();
17    }
18 }
```

The 'Run' window at the bottom shows the execution output:

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50416" -Dfile.encoding=UTF-8
Enter a string: 121
Palindrome
Process finished with exit code 0
```

The status bar at the bottom indicates the file is in 'src > PalindromeChecker' with encoding 'UTF-8' and 4 spaces.

Class and Object Creation



The screenshot shows the IntelliJ IDEA interface with a project named 'upskilling'. The 'src' directory contains several Java files, including 'CarDemo.java'. The code in 'CarDemo.java' defines a 'Car' class with a 'displayDetails()' method and a 'CarDemo' class with a 'main()' method. The 'main()' method creates a 'Car' object, sets its 'make' to 'Toyota' and 'model' to 'Camry', and then calls 'displayDetails()'. The 'displayDetails()' method prints the model and year. The 'Run' tab shows the execution output, which displays the model and year for the Toyota Camry.

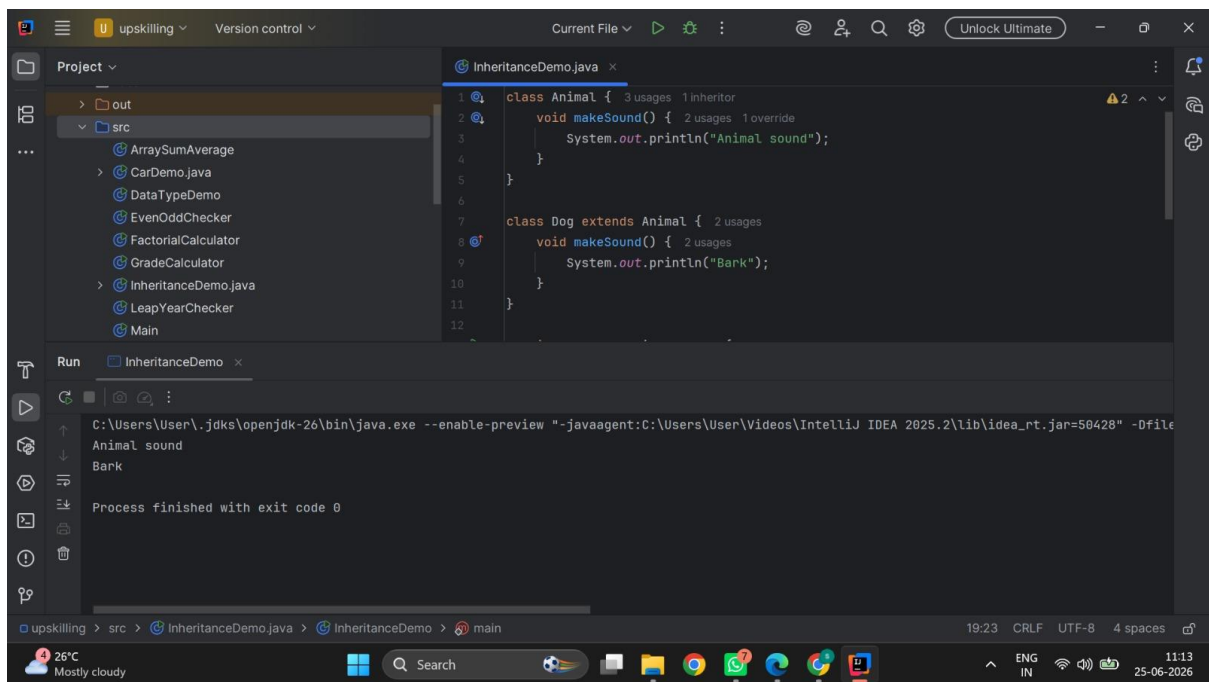
```
1 class Car { 4 usages
2     void displayDetails() { 2 usages
3         System.out.println("Model: " + model);
4         System.out.println("Year: " + year);
5     }
6 }
7
8 public class CarDemo {
9     public static void main(String[] args) {
10         Car car1 = new Car();
11         car1.make = "Toyota";
12         car1.model = "Camry";
13     }
14 }
```

Run: CarDemo

```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50422" -Dfile.encoding=UTF-8
Make: Toyota
Model: Camry
Year: 2023
Make: Honda
Model: City
Year: 2022

Process finished with exit code 0
```

Inheritance Example



The screenshot shows the IntelliJ IDEA interface with a project named 'upskilling'. The 'src' directory contains several Java files, including 'InheritanceDemo.java'. The code in 'InheritanceDemo.java' defines an 'Animal' class with a 'makeSound()' method and a 'Dog' class that extends 'Animal' and overrides 'makeSound()' to print 'Bark'. The 'main()' method creates an 'Animal' object and a 'Dog' object, and calls 'makeSound()' on both. The 'Run' tab shows the execution output, which displays 'Animal sound' and 'Bark'.

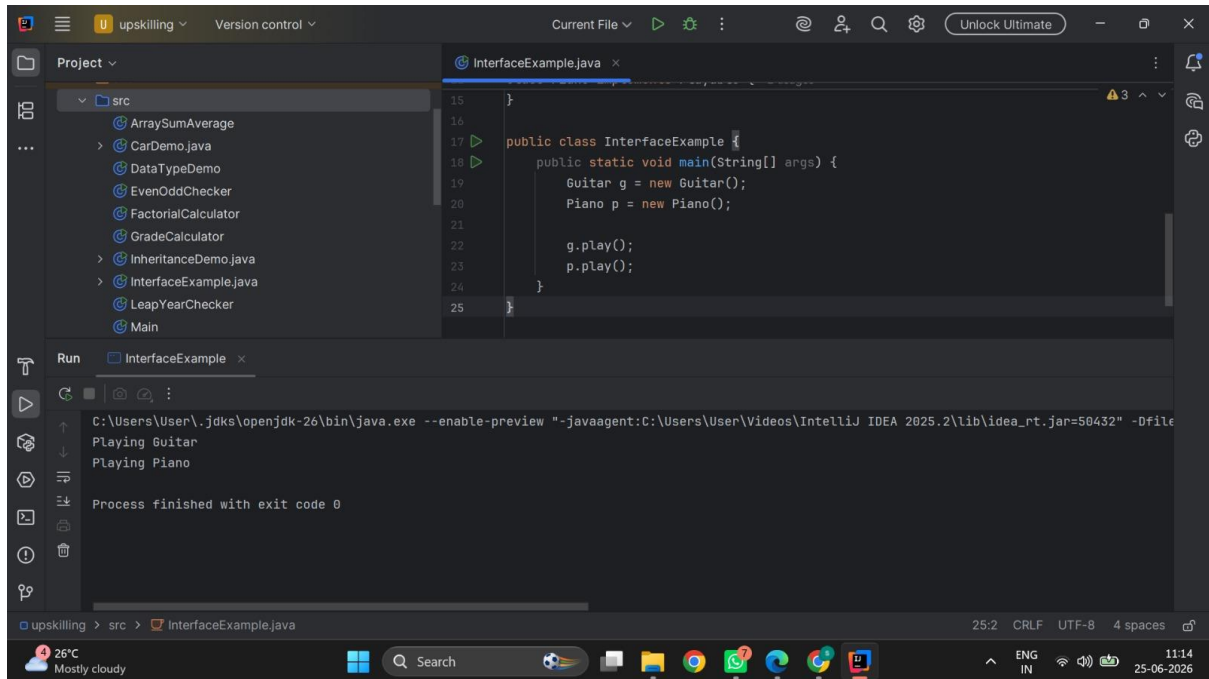
```
1 class Animal { 3 usages 1 inheritor
2     void makeSound() { 2 usages 1 override
3         System.out.println("Animal sound");
4     }
5 }
6
7 class Dog extends Animal { 2 usages
8     void makeSound() { 2 usages
9         System.out.println("Bark");
10    }
11 }
12
```

Run: InheritanceDemo

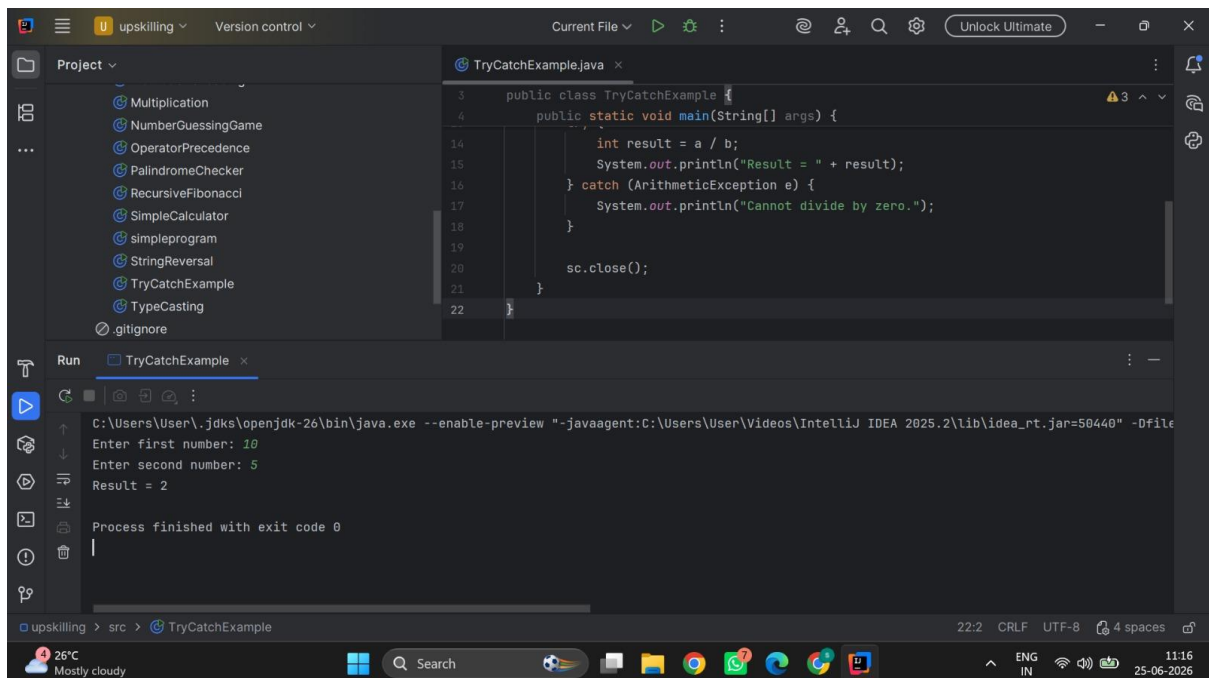
```
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50428" -Dfile.encoding=UTF-8
Animal sound
Bark

Process finished with exit code 0
```

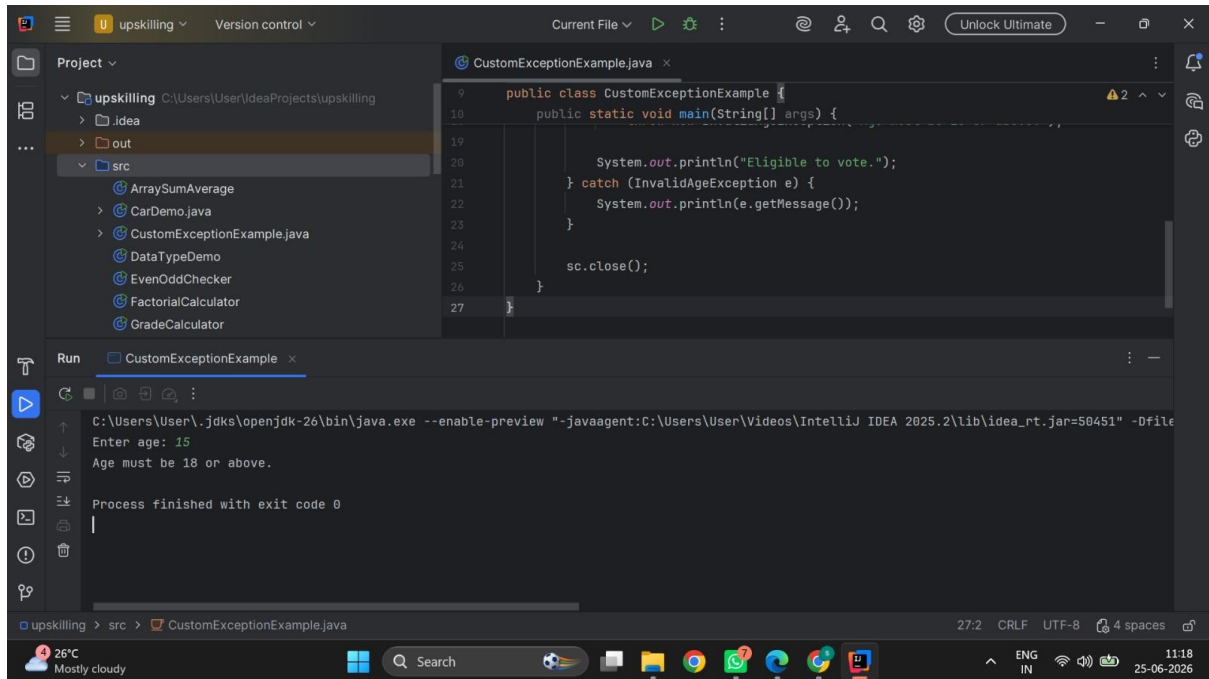
Interface Implementation



Try-Catch Example



Custom Exception

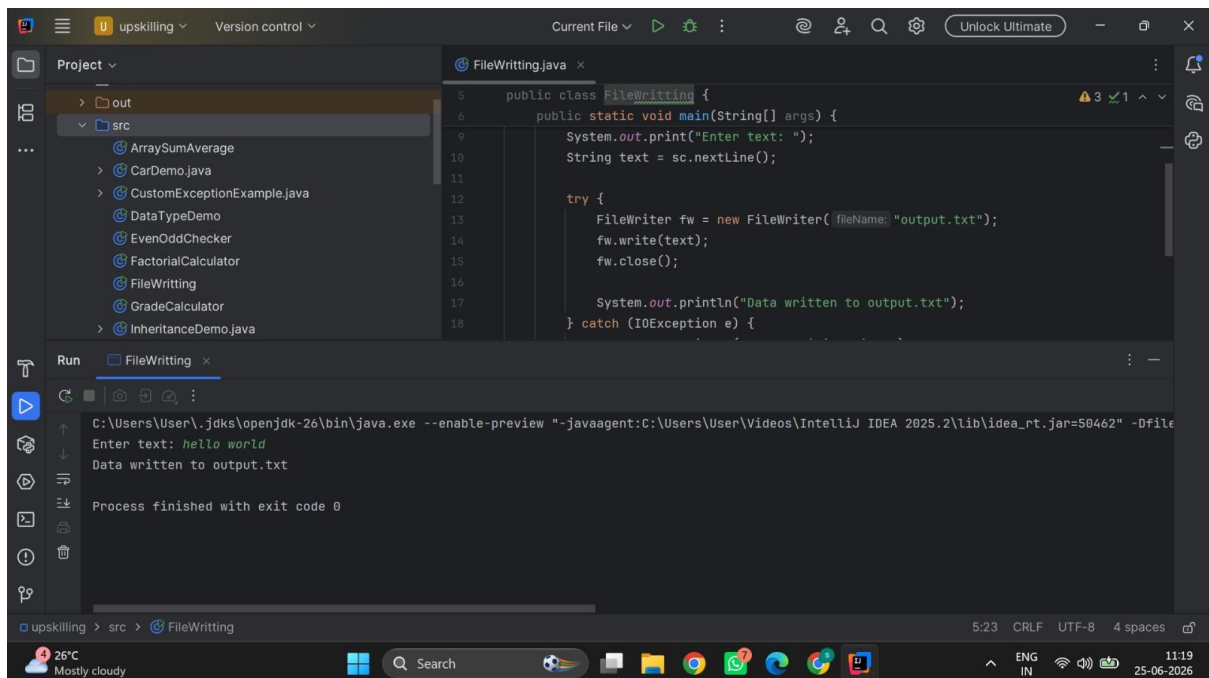


```
9 public class CustomExceptionExample {
10     public static void main(String[] args) {
11         Scanner sc = new Scanner(System.in);
12         System.out.println("Enter age:");
13         int age = sc.nextInt();
14         if (age < 18) {
15             throw new InvalidAgeException("Age must be 18 or above.");
16         }
17         System.out.println("Eligible to vote.");
18     }
19 }
20
21 class InvalidAgeException extends Exception {
22     InvalidAgeException(String message) {
23         super(message);
24     }
25 }
26
27
```

Run CustomExceptionExample

```
C:\Users\User\jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50451" -Dfile.encoding=UTF-8
Enter age: 15
Age must be 18 or above.
Process finished with exit code 0
```

File Writing

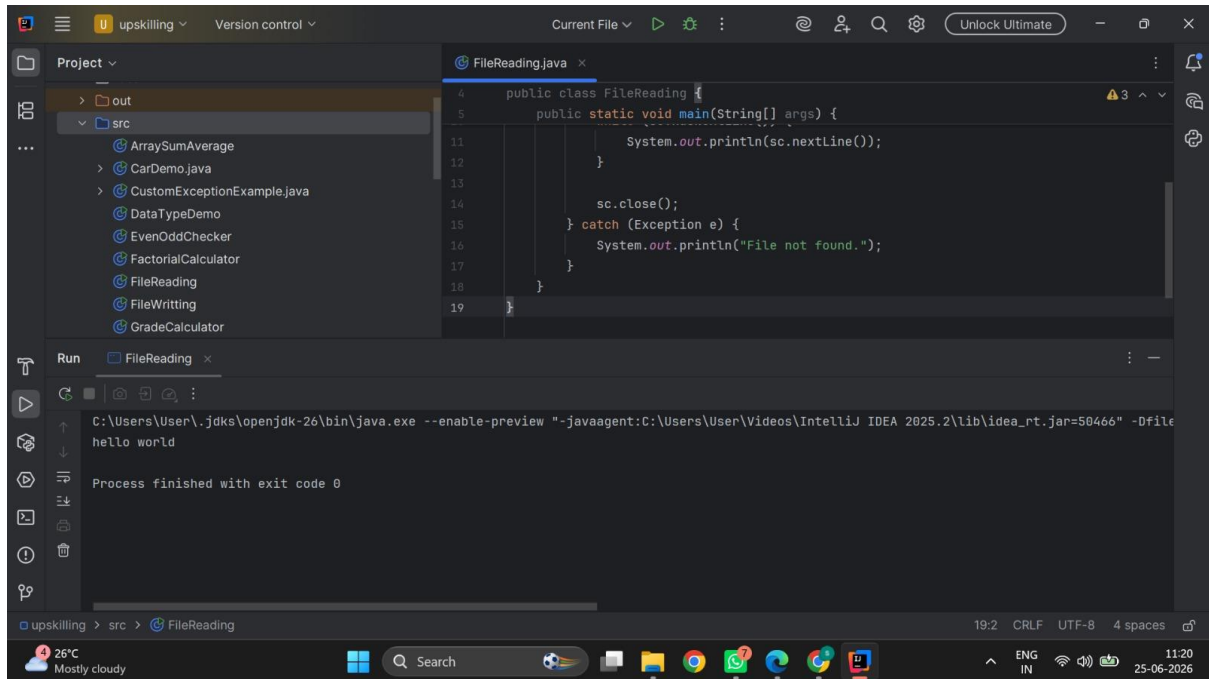


```
5 public class FileWriting {
6     public static void main(String[] args) {
7         Scanner sc = new Scanner(System.in);
8         System.out.print("Enter text: ");
9         String text = sc.nextLine();
10
11         try {
12             FileWriter fw = new FileWriter("output.txt");
13             fw.write(text);
14             fw.close();
15
16             System.out.println("Data written to output.txt");
17         } catch (IOException e) {
18             e.printStackTrace();
19         }
20     }
21 }
```

Run FileWriting

```
C:\Users\User\jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50462" -Dfile.encoding=UTF-8
Enter text: hello world
Data written to output.txt
Process finished with exit code 0
```

File Reading

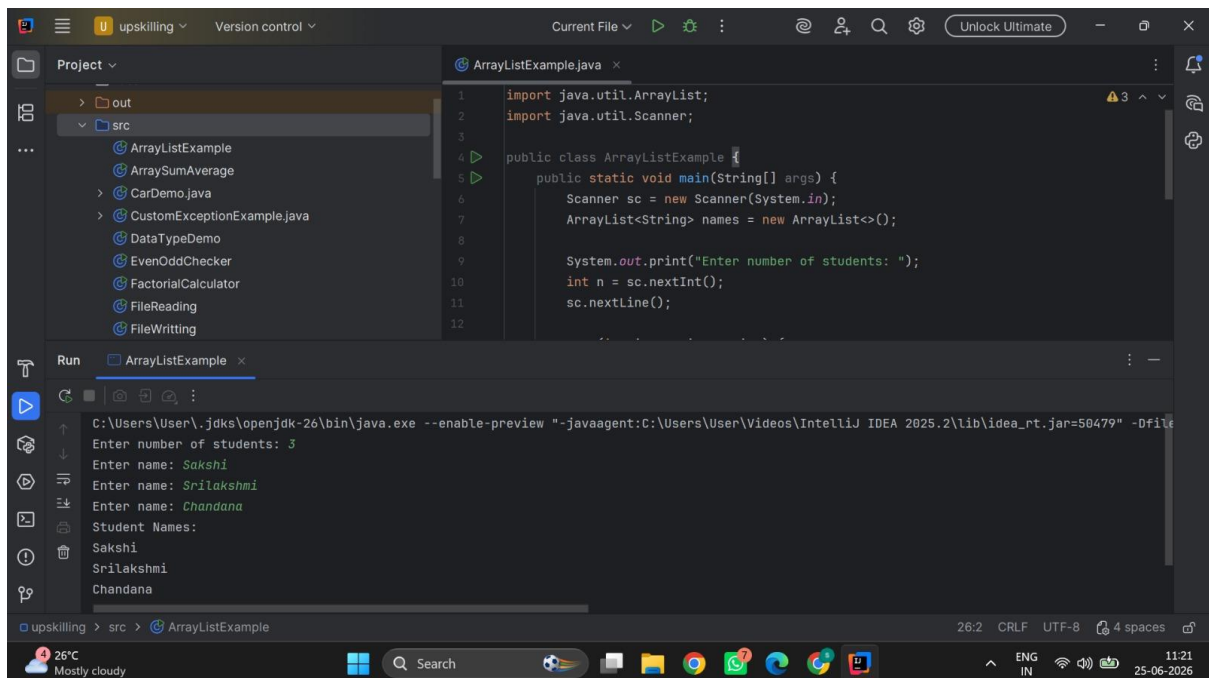


The screenshot shows the IntelliJ IDEA IDE with a project named 'upskilling'. The 'Project' view on the left shows a 'src' folder containing several Java files, including 'FileReading.java'. The 'FileReading.java' file is open in the editor, showing the following code:

```
4 public class FileReading {
5     public static void main(String[] args) {
11         System.out.println(sc.nextLine());
12     }
13
14     sc.close();
15 } catch (Exception e) {
16     System.out.println("File not found.");
17 }
18 }
19 }
```

The 'Run' view at the bottom shows the execution of the 'FileReading' class. The command line is: `C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50466" -Dfile`. The output is: `hello world`. The process finished with exit code 0.

ArrayList Example

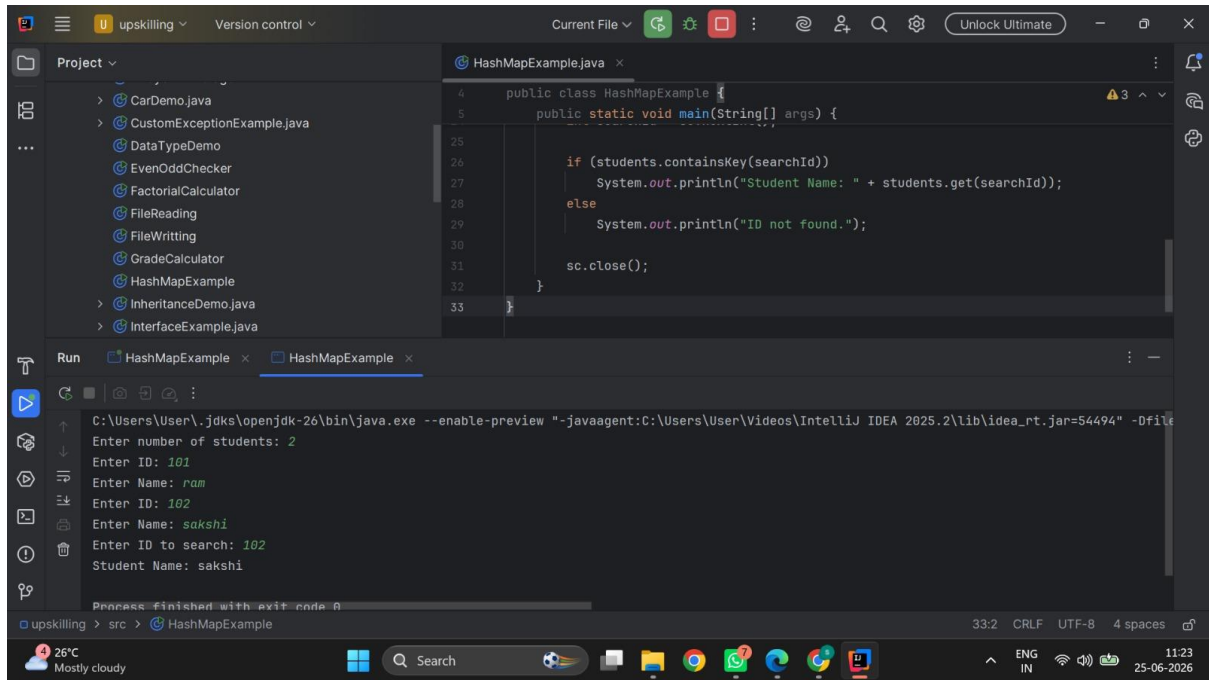


The screenshot shows the IntelliJ IDEA IDE with a project named 'upskilling'. The 'Project' view on the left shows a 'src' folder containing several Java files, including 'ArrayListExample.java'. The 'ArrayListExample.java' file is open in the editor, showing the following code:

```
1 import java.util.ArrayList;
2 import java.util.Scanner;
3
4 public class ArrayListExample {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7         ArrayList<String> names = new ArrayList<>();
8
9         System.out.print("Enter number of students: ");
10        int n = sc.nextInt();
11        sc.nextLine();
12    }
```

The 'Run' view at the bottom shows the execution of the 'ArrayListExample' class. The command line is: `C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=50479" -Dfile`. The output is: `Enter number of students: 3`, `Enter name: Sakshi`, `Enter name: Srilakshmi`, `Enter name: Chandana`, `Student Names:`, `Sakshi`, `Srilakshmi`, `Chandana`.

HashMap Example



```
Project
├── CarDemo.java
├── CustomExceptionExample.java
├── DataTypeDemo
├── EvenOddChecker
├── FactorialCalculator
├── FileReading
├── FileWriting
├── GradeCalculator
├── HashMapExample
├── InheritanceDemo.java
└── InterfaceExample.java
```

```
HashMapExample.java
4 public class HashMapExample {
5     public static void main(String[] args) {
25         if (students.containsKey(searchId))
26             System.out.println("Student Name: " + students.get(searchId));
27         else
28             System.out.println("ID not found.");
29
30         sc.close();
31     }
32 }
33 }
```

```
Run
C:\Users\User\.jdk\openjdk-26\bin\java.exe --enable-preview "-javaagent:C:\Users\User\Videos\IntelliJ IDEA 2025.2\lib\idea_rt.jar=54494" -Dfile.encoding=UTF-8
Enter number of students: 2
Enter ID: 101
Enter Name: ram
Enter ID: 102
Enter Name: sakshi
Enter ID to search: 102
Student Name: sakshi
Process finished with exit code 0
```

33:2 CRLF UTF-8 4 spaces

26°C Mostly cloudy 11:23 25-06-2026